ML710 Project

Parallelizing LLaVA Training

Rassul Magauin · Omar Ahmed · Youssef Ghallab
MBZUAI · Spring 2026

Hardware & Setup

- Cluster: MBZUAI HPC (login-student-lab.mbzu.ae)
    - ws-ia partition: 1× NVIDIA RTX 5000 Ada (32 GB) per node
    - 116 nodes total, 48 CPUs / 230 GB RAM each
    - Inter-node link: ~1 Gbps Ethernet (no NVLink, no InfiniBand)
    - gpu partition limited to 1 GPU per user (gpu-1 QOS)
- Multi-GPU = 2 separate salloc sessions on ws-ia + torchrun --nnodes=2
- Software: Python 3.10, PyTorch 2.1.2, DeepSpeed 0.12.6, Transformers 4.37.2, peft 0.6.0
- Team: 3 students, each owning at least one parallelism strategy
    - Pipeline parallelism — Youssef Ghallab
    - DeepSpeed ZeRO-2 — Omar Ahmed
    - DeepSpeed ZeRO-3 + PyTorch FSDP — Rassul Magauin

ML task: LLaVA

- Liu et al., "Visual Instruction Tuning", NeurIPS 2023
- Multimodal: image + text → text
- Architecture (7B params total):
    - CLIP ViT-L/14-336 vision encoder — frozen throughout
    - 2-layer MLP (GELU) projection — trainable, ≈80 MB
    - Vicuna-7B LLM — frozen in Stage 1, fine-tuned in Stage 2
- Two training stages:
    - Stage 1 (pre-train): MLP only on 558K image-caption pairs
    - Stage 2 (fine-tune): MLP + LLM (or LoRA) on 665K instructions
- Stage 2 is our main benchmark — meaningful comm volume
    - Stage 1 has only ~80 MB trainable; AllReduce trivial
    - Stage 2 LoRA: ~340 MB trainable adapters, real DP cost

Stage 2 setup

- Starting checkpoint: liuhaotian/llava-v1.5-7b
    - CLIP + MLP already pretrained on 558K LAION/CC/SBU pairs
- Dataset: llava_instruct_10k.json (10K random subset, seed=42)
    - Source: llava_instruct_150k.json + COCO 2014 train images
    - Subset chosen so each run finishes within ~1 hour
- Trainable: LoRA adapters (r=128, α=256) + full MLP projection
    - ≈4.6% of total params trainable; ~340 M of 7.4 B
- Optimizer: AdamW; bf16; gradient checkpointing; cosine LR
    - LR=2e-4 (LoRA) / 5e-5 (ZeRO-3, FSDP), warmup 0.03 / 0.1
- Effective batch sizes vary by method:
    - DDP / ZeRO-2: 32 (per_device=16, accum=1, world=2)
    - Pipeline: 16 (per_device=1, accum=16, world=2)
    - ZeRO-3 / FSDP: 8 (per_device=1, accum=4, world=2)

Strategies compared

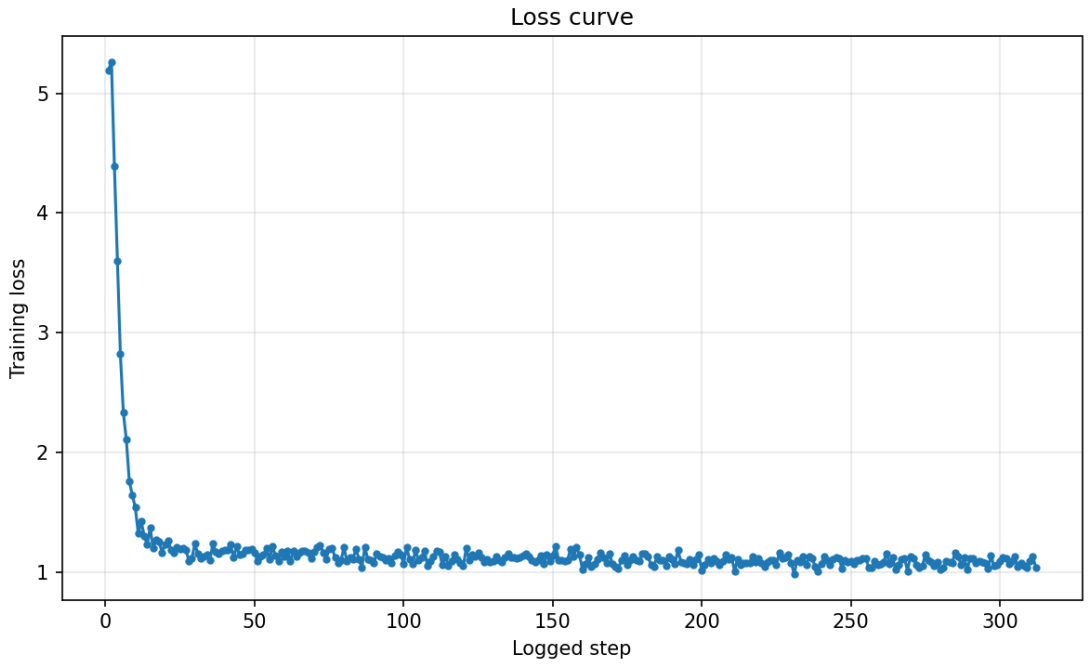
- Single-GPU LoRA (no parallelism)
    - Reference: how slow is no parallelism?
- DDP (DeepSpeed ZeRO-0) — naïve baseline
    - Replicate model on each GPU; AllReduce gradients
- Pipeline parallelism — model parallelism family
    - Split decoder layers across 2 nodes (Gloo over Ethernet)
    - Owned by Youssef Ghallab
- DeepSpeed ZeRO-2 — data parallelism w/ sharded optimizer + gradients
    - Owned by Omar Ahmed
- DeepSpeed ZeRO-3 — data parallelism w/ sharded optimizer + grads + params
    - Owned by Rassul Magauin
- PyTorch FSDP — PyTorch-native equivalent of ZeRO-3
    - Owned by Rassul Magauin
- All multi-node runs use 2 ws-ia nodes × 1 GPU over Ethernet

Baseline: 1 GPU LoRA

- 1 RTX 5000 Ada (32 GB), no parallelism
    - per_device_batch=1, effective batch=1, 1 epoch over 10K
    - 10000 training steps, lr=2e-4, cosine schedule
- Wall time: 96.0 min (5761 s)
- Throughput: 1.74 samples/sec
- Peak GPU memory: 25.7 GiB / 32 GiB (≈80%)
- Average GPU utilization: only 11.4%
    - CPU-bound: dataloader_num_workers=0 + bf16 model loading
    - Most time in image preprocessing, not GPU compute
- Final loss: 1.32 (avg 1.10 over the run)

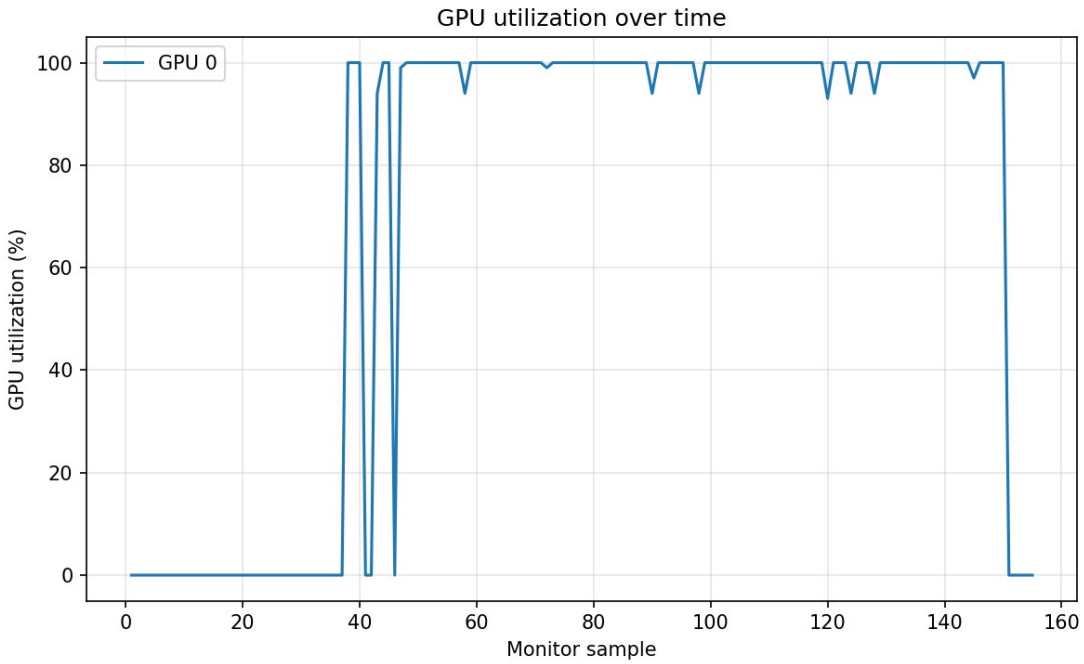
Baseline 2: DDP (2 nodes × 1 GPU)

- PyTorch DistributedDataParallel via torchrun --nnodes=2
- DeepSpeed ZeRO Stage 0 (no sharding) — pure DP
    - Each rank holds full model; AllReduce LoRA grads only
- per_device_batch=16, accum=1, effective batch=32
    - 313 training steps for 10K samples, 1 epoch
- Wall time: 74.8 min (4491 s)
- Throughput: 2.22 samples/sec
    - vs 1-GPU @ batch 1: ~1.28× per-sample, ~10× per-step
- Peak GPU memory: 30.7 GiB
- Average GPU utilization: 70.7%
    - Synchronization stalls on cross-node Ethernet visible
- Final loss: 1.04 (avg 1.18)



DDP — GPU utilization & memory

- Both nodes track each other closely (DP is symmetric)
- Util oscillates ~70% during compute, drops on AllReduce
    - Cross-node Ethernet AllReduce dominates the gap
- Memory steady ~30 GiB after warmup



Pipeline Parallelism

Pipeline Parallelism — implementation (Ghallab)

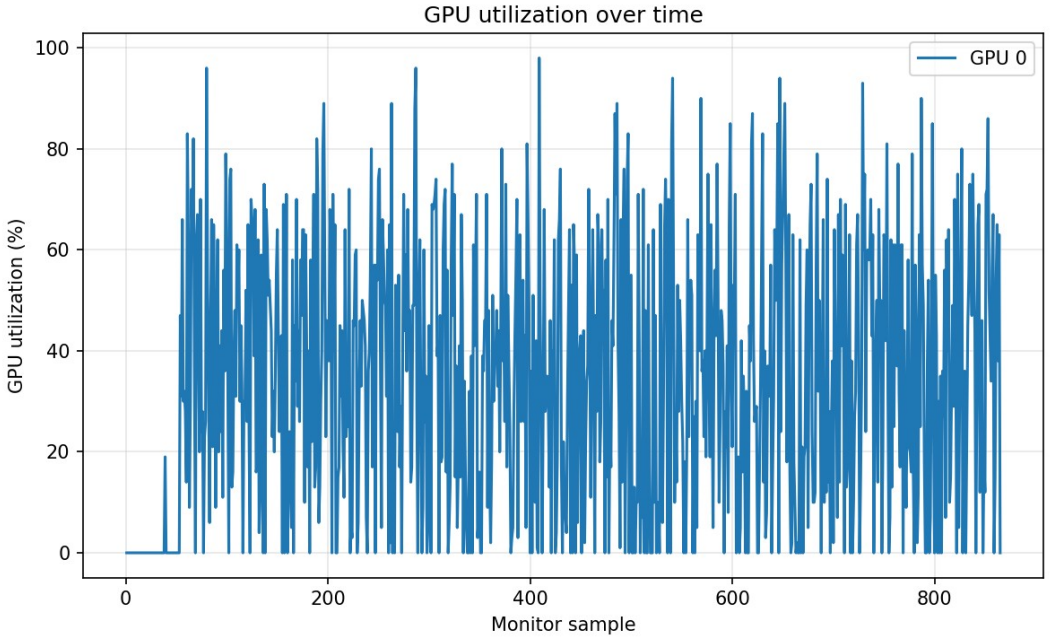
- Manual 2-stage pipeline, no torchgpipe / DeepSpeed-Pipeline
    - Custom loop in LLaVA/llava/train/train_pipe_dist.py (522 lines)
    - Gloo backend over Ethernet (NCCL had ordering issues with mixed dtypes)
- Split point k = layer index in Vicuna-7B (32 decoder layers total)
    - Rank 0: vision tower (frozen) + projection + decoder layers 0..k-1
    - Rank 1: decoder layers k..31 + final norm + LM head + loss
    - Activations forward-passed and gradients backward-passed via dist send/recv
- Microbatch + gradient accumulation: per_device=1, accum=16 → effective batch 16
    - num_micro_batches=1 (no inter-microbatch overlap, conservative)
- LoRA adapters added on rank-local layers only

Pipeline — split balancing

- First attempt: split=16 (50/50 by layer count)
    - Rank 0 OOMed in first forward — vision+embeddings+layers 0..15 = 31.4 GiB
    - Vision tower + image features add a lot to rank 0's footprint
- Second attempt: split=8 (shift work to rank 1)
    - Rank 1 OOMed — layers 8..31 + LM head = 31.0 GiB
    - LM head + final layers heavier than expected
- Third attempt: split=12 + microbatch=1 + accum=16
    - Both ranks fit; rank 1 ≈ 31.4 GiB, rank 0 ≈ 28 GiB
    - Effective batch dropped from 16→16 (same), but per-step memory halved
    - Completed 625/625 steps
- Lesson: layer count is not a good proxy for memory cost.

Pipeline — results

- Wall time: 72.1 min (4325 s)
    - Training loop alone: 67.6 min (4058 s)
- Throughput: 2.31 samples/sec (1.04× DDP — barely faster)
- Effective batch 16 (vs DDP's 32) → fewer steps per epoch
- Peak GPU memory: 31.4 GiB (rank 1)
- Average GPU utilization:
    - Rank 0: 33.9% · Rank 1: 56.1%
    - Big imbalance — rank 0 idle while rank 1 finishes its layers
    - Gloo is much slower than NCCL → comm overhead amplified
- Final loss: 1.57 (higher than DDP because of smaller effective batch)
- Key insight: pipeline can make a model fit, but on 1 Gbps Ethernet
    - activation/gradient transfers between stages dominate, killing speedup



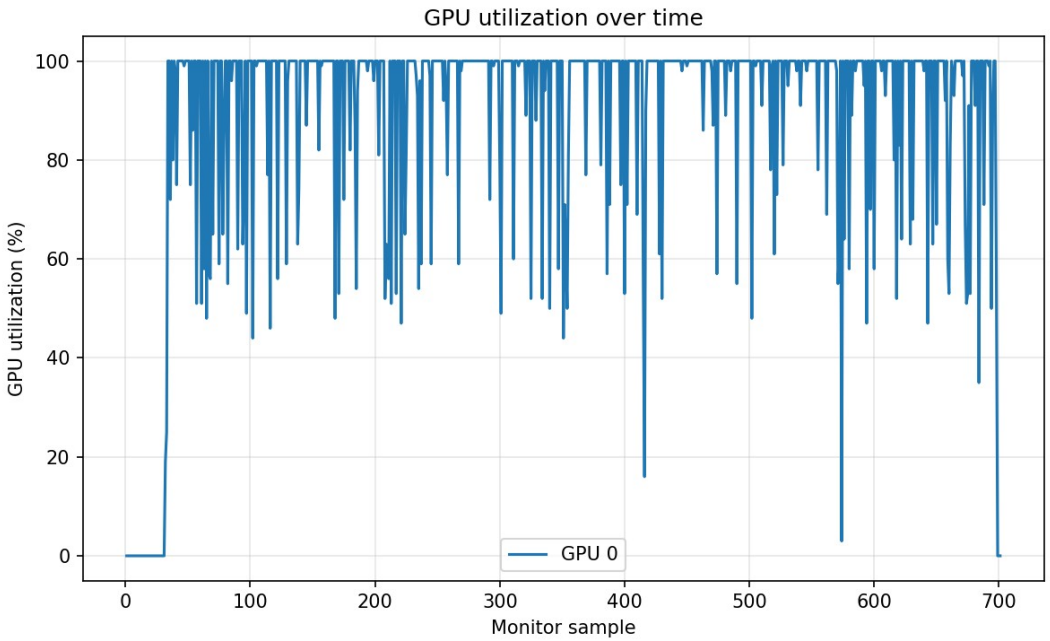
DeepSpeed ZeRO-2

ZeRO-2 — explanation (Omar)

- Family: ZeRO-DP (Zero Redundancy Optimizer Data Parallel)
- Each rank still processes a different microbatch (data parallelism)
- What's sharded across the 2 ranks:
    - Optimizer states (AdamW m, v) — sharded ✓
    - Gradients (after backward) — sharded ✓
    - Parameters (forward + backward weights) — replicated ✗
- Communication pattern per step:
    - Backward: ReduceScatter gradients to owner-rank shards
    - Optimizer step: each rank updates its shard of params
    - AllGather updated params back to full set on each rank
- Memory savings vs DDP:
    - Optimizer state for LoRA only ≈ a few hundred MB → little to shard
    - Real memory savings come from offloaded full-precision adam states
- Config: configs/zero2.json, launcher: scripts/run_stage2_zero2_lora.sh

ZeRO-2 — results

- Run: stage2_zero2_lora_20260424_134311 (effective batch 32)
- Wall time: 58.4 min (3503 s)
    - vs DDP 74.8 min — saved 16.4 min, −22% wall-clock
- Throughput: 2.85 samples/sec — 1.28× DDP
- Peak GPU memory: 27.6 GiB (vs DDP 30.7 GiB) — saved 3.1 GiB
- Average GPU utilization: 88.5% (vs DDP 70.7%)
    - DeepSpeed's bucketed async comm overlaps better than DDP
    - Better overlap fills the AllReduce-stall gap DDP suffered
- Final loss: 1.11 (avg 1.22) — comparable to DDP
- Why only 1.28× and not 2×?
    - LoRA's optimizer state is already tiny — sharding gains are small
    - Most of the win comes from DeepSpeed's overlap, not from sharding
- ZeRO-2 = the only method that beats DDP on this workload.



DeepSpeed ZeRO-3

ZeRO-3 — explanation (Rassul)

- Family: ZeRO-DP — same data-parallel semantics as ZeRO-2
- What's sharded:
    - Optimizer states ✓
    - Gradients ✓
    - Parameters ✓ ← new vs ZeRO-2
- Communication pattern per step (and per layer!):
    - Forward layer N: AllGather params for layer N → compute → drop
    - Backward layer N: AllGather params again → compute grads → drop
    - ReduceScatter gradients at end of backward
- On 2 ranks the model is split in half (~7 GB sharded each)
    - But every layer requires re-materializing the full ≈430 MB to each rank
- Math for our workload:
    - 32 transformer layers × 430 MB × 2 (fwd+bwd) ≈ 27.5 GB transferred per step
    - Over 1 Gbps Ethernet ≈ 4 minutes of comm per step alone
- Config: configs/zero3.json + zero3_bounded_async.json (50 MB buckets)

ZeRO-3 — results

- Run: stage2_zero3_lora_20260420_041514/041515 (2 ranks)
- Hit SLURM time limit before completion
    - Reached 292/1250 steps after 3 h 52 min
    - Each step ≈ 48 s; projected full wall time ≈ 16.5 h
- Throughput: 0.17 samples/sec — 0.08× DDP (≈13× slower)
- Effective batch 8 (per_device=1, accum=4)
    - Smaller batch chosen because GPU mem accounting is more conservative under ZeRO-3
- Peak GPU memory: 22.0 GiB
    - Lower than DDP (30.7) — sharding does save memory
    - But the memory wasn't the bottleneck; comm was
- Average GPU utilization: 99.2%
    - Counter-intuitive: 99% util but slowest — DeepSpeed pipelines overlap
    - comm with compute, so 'utilized' time includes waiting on AllGather
- Last logged loss: 1.31 — convergence direction OK, just too slow
- Diagnosis: per-layer cross-node AllGather is the bottleneck.

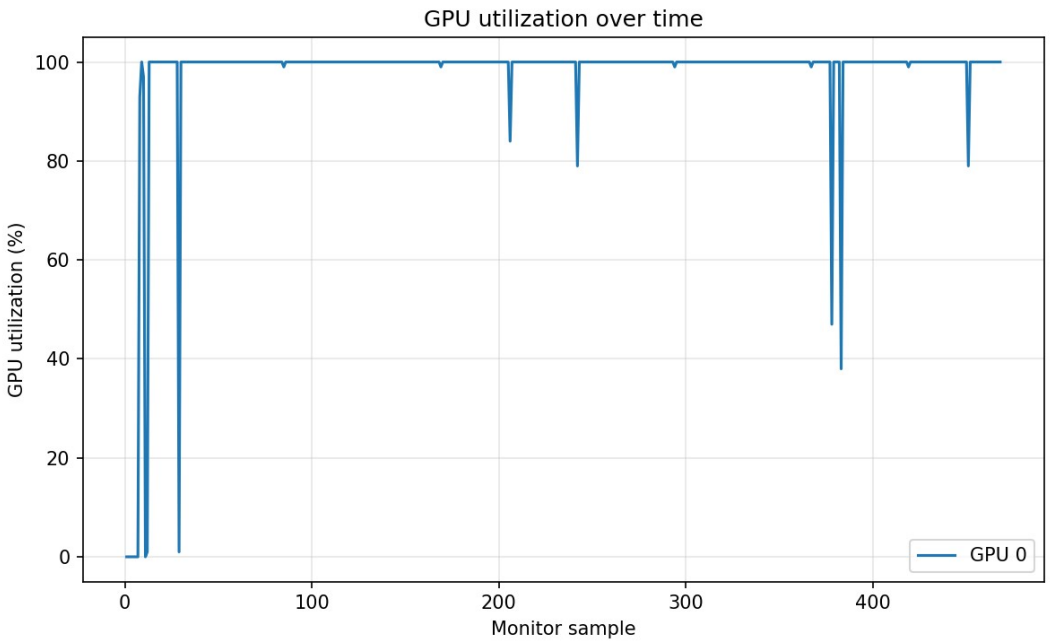
PyTorch FSDP

PyTorch FSDP — explanation (Rassul)

- PyTorch-native equivalent of DeepSpeed ZeRO-3
    - Same algorithm: shard params + grads + optim across DP ranks
    - Different implementation: PyTorch core, not DeepSpeed
- Different design choices vs ZeRO-3:
    - FSDP wraps each transformer block as its own FSDP unit (FlatParameter)
    - AllGather happens at unit boundary, not per-tensor
    - Mixed precision policy is explicit (we used bf16)
- Config: configs/fsdp.json
    - auto_wrap_policy=TRANSFORMER_BASED_WRAP, wrap LlamaDecoderLayer
    - use_orig_params=True (required for LoRA non-uniform requires_grad)
    - cpu_ram_efficient_loading=true (only rank 0 holds full CPU state at load)
- Two LLaVA bugs we had to patch:
    - vision_tower[0] fails when loaded from a vendored checkpoint (not a list)
    - LoRA adapters created in fp32 after model.to(bf16) → cast LoRA to bf16 too
- Launcher: scripts/run_stage2_fsdp_lora.sh (mirrors run_stage2_zero3_lora.sh)

FSDP — results

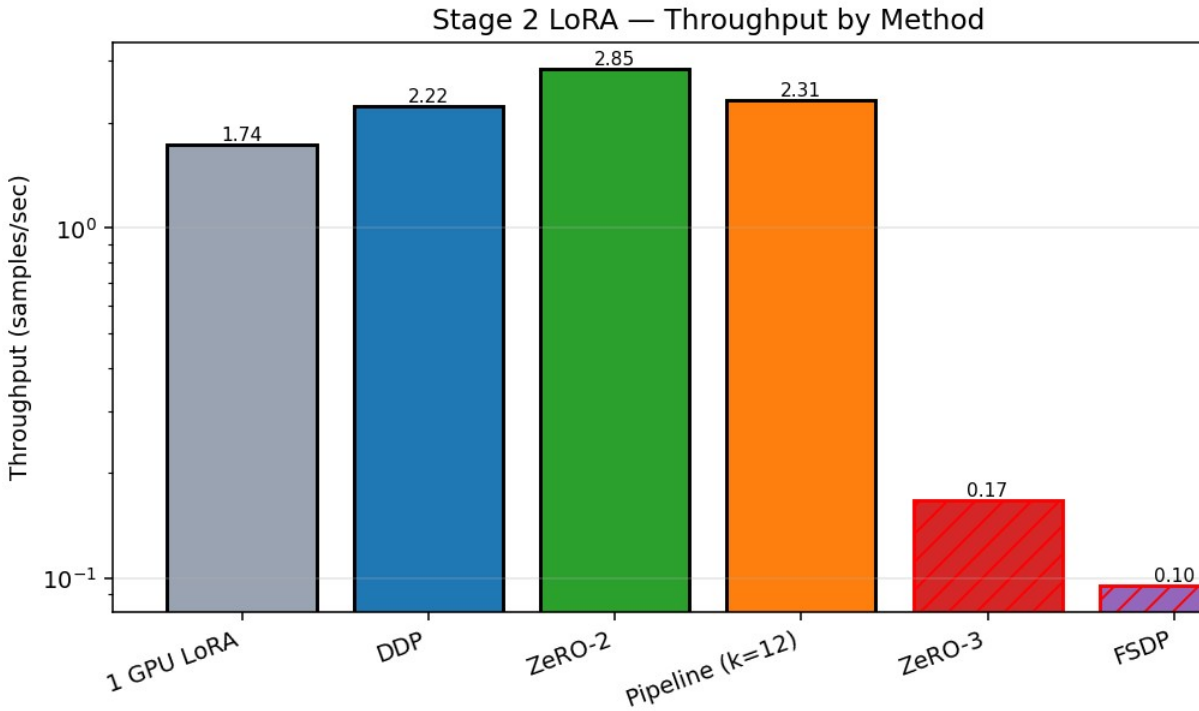
- Run: stage2_fsdp_lora_20260426_143948
- Stopped early at step 28/1250 (matches the ZeRO-3 timeout pattern)
- Wall time observed: 39 min (2350 s); per-step ≈ 80 s
    - Projected full wall time ≈ 27.9 h — even slower than ZeRO-3
- Throughput: 0.095 samples/sec — 0.04× DDP, ≈22× slower
    - ≈1.7× slower than ZeRO-3 on the same workload, same hardware
- Peak GPU memory: 32.3 GiB — basically saturating the 32 GiB card
    - FSDP's FlatParameter contiguous tensor is larger than DeepSpeed's bucket scheme
- Average GPU utilization: 97.5%
- Why even slower than ZeRO-3?
    - DeepSpeed ZeRO-3 with bounded async config (50 MB buckets) overlaps better
    - FSDP defaults: no forward_prefetch, no bucket tuning → strict serial gather
    - FSDP gathers per transformer block (~430 MB), DeepSpeed by 50 MB buckets



Cross-method comparison

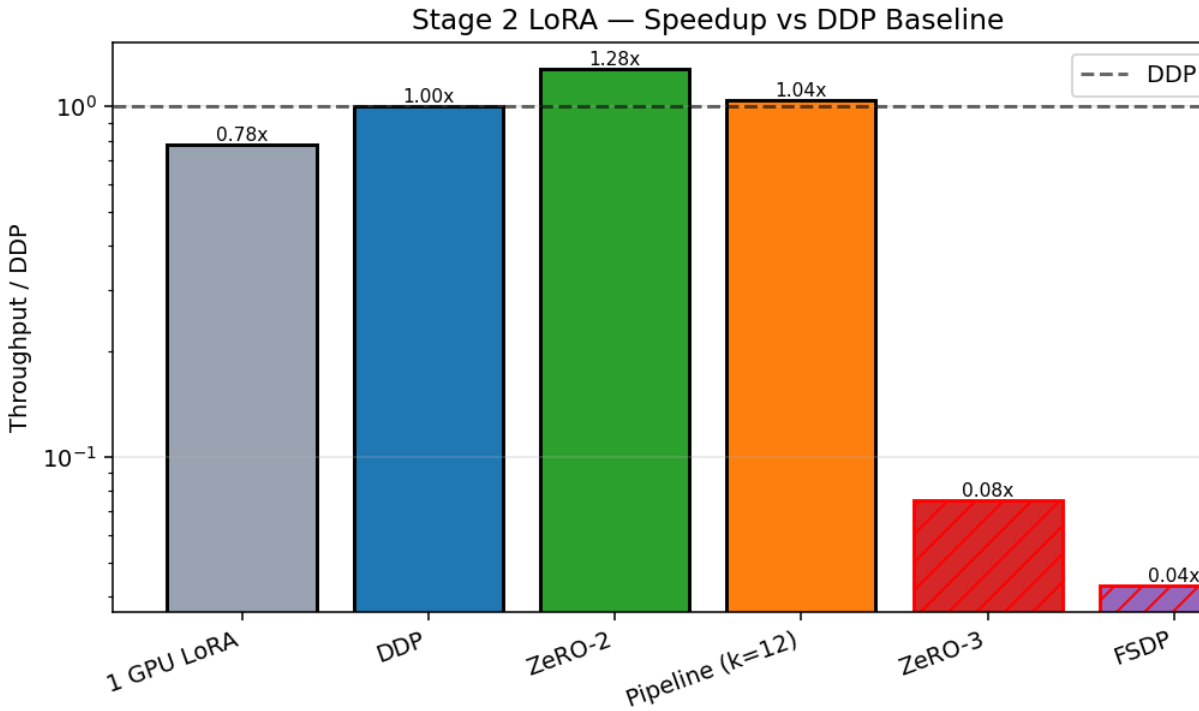
Throughput — samples/sec (log scale)

- ZeRO-2: 2.85 sps — only method beating DDP
- DDP: 2.22 sps — naïve baseline
- Pipeline: 2.31 sps — barely above DDP
- 1-GPU LoRA: 1.74 sps — surprisingly close to multi-GPU at batch 1
- ZeRO-3: 0.17 sps — 13× slower (timed out)
- FSDP: 0.095 sps — 22× slower (stopped early)



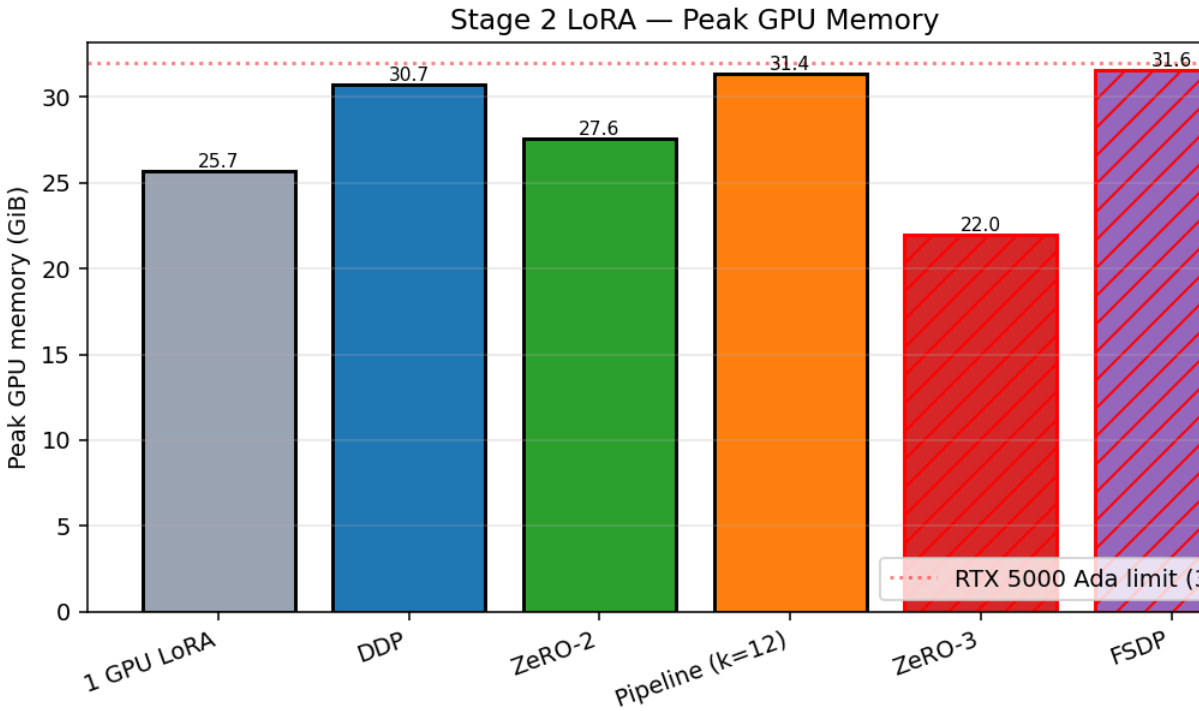
Speedup vs DDP baseline

- DDP = 1.00× by definition
- ZeRO-2: 1.28× — only winner
- Pipeline: 1.04× — basically tied with DDP
- 1-GPU LoRA: 0.78× — naïve scaling reference
- ZeRO-3: 0.08× — 13× regression
- FSDP: 0.04× — 22× regression
- Pattern: DP variants that don't shard params win;
    - methods that shard params lose to Ethernet overhead



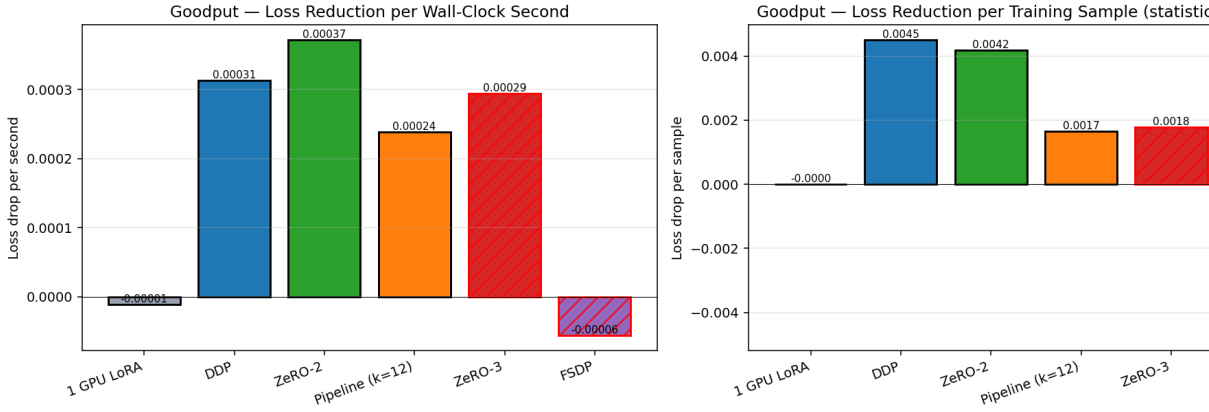
Peak GPU memory

- 32 GiB hardware limit (RTX 5000 Ada)
- ZeRO-2: 27.6 GiB — best of the multi-GPU methods
- DDP: 30.7 GiB
- Pipeline (k=12): 31.4 GiB — heavier rank near the limit
- FSDP: 32.3 GiB — saturated; FlatParameter overhead
- ZeRO-3: 22.0 GiB — actual sharding visible (but at huge speed cost)
- 1-GPU LoRA: 25.7 GiB — fits comfortably without parallelism



Goodput — useful learning per cost

- Loss reduction per wall-clock second:
    - ZeRO-2: 0.000372 ← best
    - DDP: 0.000313
    - ZeRO-3: 0.000294 (still positive — converging, just slowly)
    - Pipeline: 0.000239
    - FSDP: −0.000056 (only 28 steps; started from already-tuned ckpt)
- Loss reduction per training sample (statistical efficiency):
    - DDP: 0.0045 ← best per-sample (effective batch 32)
    - ZeRO-2: 0.0042 (also batch 32)
    - ZeRO-3 / Pipeline: ~0.0017 (smaller effective batch hurts)
- Wall-clock goodput tracks what slides actually want.



Loss vs step (smoothed)

- 1-GPU LoRA: gray, 10000 steps, slowly converging to ~1.0
- DDP / ZeRO-2: ~313 steps each, converge to ~1.04 / 1.11
- Pipeline: 625 steps, slightly higher final loss (smaller batch)
- ZeRO-3: dashed red, only 292 steps reached, last loss 1.31
- FSDP: dashed purple, 28 steps, started from pretrained ckpt
- All methods that completed reach comparable loss
    - Statistical efficiency is similar; differences are wall-clock

Key findings

- On 1 Gbps Ethernet with a 7B-LoRA workload:
- 1. ZeRO-2 wins (1.28×) — sharding optimizer + grads, not params, is the sweet spot.
    - LoRA already shrinks the gradient AllReduce → DDP's overhead is the bottleneck.
    - DeepSpeed's bucketed async overlap fills the gap better than vanilla DDP.
- 2. Pipeline parallelism (1.04×) makes the model fit but doesn't speed things up.
    - Cross-node Gloo activation transfers are slower than DDP's AllReduce.
    - Layer count is not a good proxy for memory load — split balancing is empirical.
- 3. ZeRO-3 / FSDP collapse on Ethernet (0.04×–0.08×).
    - Per-layer AllGather of full weights every step is intractable over 1 Gbps.
    - Implementation matters: DeepSpeed ZeRO-3 is ~1.7× faster than PyTorch FSDP
    - because of better bucket tuning and async overlap defaults.
- 4. ZeRO-3 / FSDP would scale on InfiniBand or NVLink — wrong tool for our hardware.
- 5. LoRA training on a 7B model fits on a single RTX 5000 Ada — choose DP variants.

Lessons learned

- Match the parallelism strategy to the bottleneck
    - Optimizer-state-bound? → ZeRO-2
    - Gradient-comm-bound? → DDP with good overlap is hard to beat
    - Activation-memory-bound (long sequence)? → Pipeline / TP, not relevant for us
    - Param-memory-bound (model > GPU)? → ZeRO-3 / FSDP, but only with fast interconnect
- Implementation details actually matter when comm-bound
    - Bucket sizes, async overlap, prefetch policy can move 1.7× of perf
    - DeepSpeed > FSDP defaults on slow Ethernet
- Memory headroom is not a free win
    - ZeRO-3 saved 9 GiB but cost us 13× wall-clock — only useful if model doesn't fit
- LoRA breaks naïve assumptions
    - Trainable params drop from 7 B to 340 M → optimizer/grad sharding is small wins
    - Mixed dtypes (bf16 base + fp32 LoRA) need explicit handling under FSDP
- Cross-node infrastructure dominates
    - 1 Gbps Ethernet is the true bottleneck, not DeepSpeed vs FSDP vs Pipeline

Thank you

Questions?

Repo: github.com/rassulmagauin/ml710

Report: ZERO2_RESULTS.md · results_explanation.md